EMERIS

ASSIGNMENT 1
PROG6212

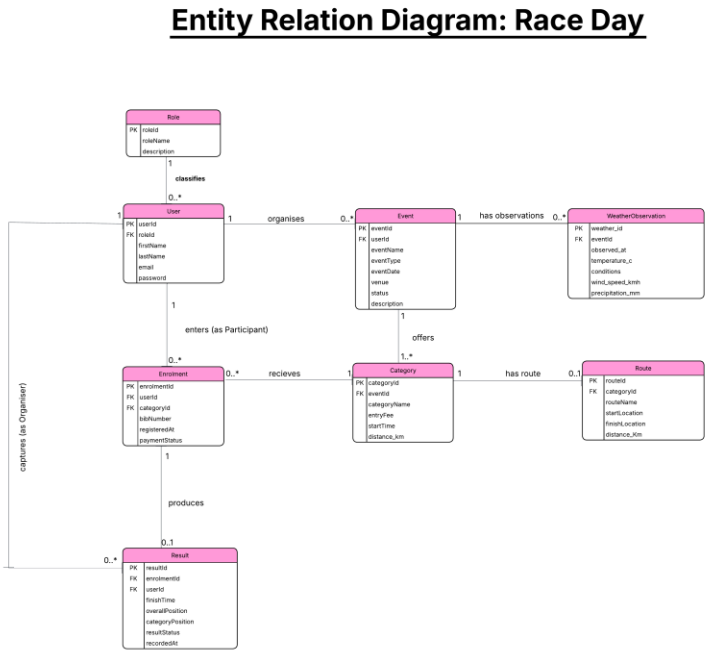
2026

Prepared By :

Bonolo Mathabela

Programming 2B
Practical Assignment 1
ST10441596

Section A



Relationship	Cardinality	Description
Role → User	1 : 0..*	A role may be held by no users or by many users, but every user holds exactly one role. The brief defines exactly two roles, Organiser and Participant.
User (Organiser) → Event	1 : 0..*	Every event is created by exactly one Organiser. Only Organisers can create, edit and delete events.
Event → Category	1 : 1..*	An event must offer at least one category and may offer many, while each category belongs to exactly one event.
Category → Enrolment	1 : 0..*	A category may have no entries or many entries, but each entry is for exactly one category. Participants enter an event by selecting a category.
User (Participant) → Enrolment	1 : 0..*	A participant may have no entries or many entries, but each entry belongs to exactly one participant. This lets a participant view their own enrolments across events.
Enrolment → Result	1:0..1	An entry has no result before race day and at most one result after it, while every result belongs to exactly one entry. Participants track their personal results through this link
User (Organiser) → Result	1 : 0..*	An Organiser may capture no results or many results, but every result is captured by exactly one Organiser. Only Organisers can capture participant results.
Category → Route	1 : 0..1	A category may have no route or exactly one route, and every route describes exactly one category. Participants use the route to prepare for race day.

Figure 1: ERD – Race Day (Lucidchart, 2026)

Section B

1. Authentication

HTTP Method	Route	Description	Role required	Request body	Expected response
GET	/api/roles	Returns the two system roles (Organiser, Participant) so the registration form can populate its role selector from the database instead of hard-coding values.	None	None	200 OK – array of { roleId, roleName, description }
POST	/api/auth/register	Creates a new user account with the selected role. Validates all fields, checks the email is unique and stores the password as a bcrypt hash (never plain text).	None	{ firstName, lastName, email, password, roleId }	201 Created – { userId, firstName, lastName, email, roleName } (no password) 400 Bad Request – missing/invalid field or unknown roleId 409 Conflict – email already registered
POST	/api/auth/login	Verifies the email and hashed password, then starts a server-side session holding userId and role for all subsequent requests.	None	{ email, password }	200 OK – { userId, firstName, lastName, email, roleName } + session cookie 400 Bad Request – missing fields 401 Unauthorized – invalid email or password
POST	/api/auth/logout	Destroys the current session and clears the session cookie so the user is fully logged out.	Any	None	204 No Content 401 Unauthorized – no active session
GET	/api/auth/me	Returns the currently logged-in user and role from the session; used by the front end to restore login state on page load and to decide which menu (Organiser/Participant) to show.	Any	None	200 OK – { userId, firstName, lastName, email, roleName } 401 Unauthorized – no active session

2. User Profile

HTTP Method	Route	Description	Role required	Request body	Expected response
GET	/api/users/me	Returns the logged-in user's own profile information (both roles).	Any	None	200 OK – { userId, firstName, lastName, email, roleName } 401 Unauthorized
PUT	/api/users/me	Updates the logged-in user's own profile details. Role cannot be changed here to prevent privilege escalation.	Any	{ firstName, lastName, email }	200 OK – updated profile 400 Bad Request – invalid field 409 Conflict – email already in use by another account
PUT	/api/users/me/password	Changes the logged-in user's password after verifying the current one; the new password is re-hashed before storing.	Any	{ currentPassword, newPassword }	204 No Content 400 Bad Request – new password fails policy 401 Unauthorized – current password incorrect

3. Events

HTTP Method	Route	Description	Role required	Request body	Expected response
GET	/api/events	Lists all events so both roles (and the public) can browse upcoming races. Supports optional filters ?eventType=Run|Walk|Cycle, ?status=Open and ?from=YYYY-MM-DD.	None	None	200 OK – array of events, each with its organiser name and category count
GET	/api/events/{id}	Returns full details for one event, including its categories, so a participant can decide which category to enter.	None	None	200 OK – event object with categories[] 404 Not Found – event does not exist
GET	/api/events/mine	Lists only the events created by the logged-in Organiser, for their management dashboard.	Organiser	None	200 OK – array of the organiser's events 403 Forbidden – caller is a Participant

POST	/api/events	Creates a new event owned by the logged-in Organiser (Event.userId is taken from the session, not the body). eventType must be Run, Walk or Cycle.	Organiser	{ eventName, eventType, eventDate, venue, status, description }	201 Created – new event with eventId 400 Bad Request – invalid eventType/status or date in the past 403 Forbidden – caller is a Participant 409 Conflict – same eventName on same eventDate already exists
PUT	/api/events/{id}	Updates an existing event. Only the Organiser who created the event may edit it (ownership check).	Organiser	{ eventName, eventType, eventDate, venue, status, description }	200 OK – updated event 400 Bad Request – invalid field 403 Forbidden – not the owning organiser 404 Not Found – event does not exist
DELETE	/api/events/{id}	Deletes an event and (via cascade) its categories, routes and weather observations. Refused if any participant is already enrolled, to protect entries and results.	Organiser	None	204 No Content 403 Forbidden – not the owning organiser 404 Not Found – event does not exist 409 Conflict – event has enrolments; cancel it instead
GET	/api/events/{id}/weather	Returns the weather observations for an event (latest first) so participants can prepare for race day. If none is stored for the last hour, fetches current conditions for the venue from the external weather API and saves a new WeatherObservation row.	None	None	200 OK – array of { weather_id, observed_at, temperature_c, conditions, wind_speed_kmh, precipitation_mm } 404 Not Found – event does not exist 502 Bad Gateway – external weather service unavailable

					(cached rows still returned if any)

4. Categories

HTTP Method	Route	Description	Role required	Request body	Expected response
GET	/api/events/{eventId}/categories	Lists the age/distance categories offered by an event (e.g. Under 20, 10km, 21km) so both roles can view what is available to enter.	None	None	200 OK – array of categories (each includes hasRoute flag) 404 Not Found – event does not exist
GET	/api/categories/{id}	Returns one category with its parent event summary and route (if defined).	None	None	200 OK – category object 404 Not Found – category does not exist
POST	/api/events/{eventId}/categories	Adds a new category to an event. Only the event's owning Organiser may define categories; category names must be unique within the event.	Organiser	{ categoryName, entryFee, startTime, distance_km }	201 Created – new category with categoryId 400 Bad Request – negative fee or non-positive distance 403 Forbidden – not the owning organiser 404 Not Found – event does not exist 409 Conflict – categoryName already exists for this event
PUT	/api/categories/{id}	Updates a category's name, fee, start time or distance.	Organiser	{ categoryName, entryFee, startTime, distance_km }	200 OK – updated category 400 Bad Request – invalid field 403 Forbidden – not the owning organiser 404 Not Found – category does not exist

DELETE	/api/categories/{id}	Removes a category (and its route). Refused if participants have already enrolled in it, and refused if it is the event's last category (an event must offer at least one).	Organiser	None	204 No Content 403 Forbidden – not the owning organiser 404 Not Found – category does not exist 409 Conflict – category has enrolments or is the event's only category

5. Routes

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/categories/{categoryId}/route	Returns the route (start, finish, distance) for a category so participants can prepare for race day.	None	None	200 OK – route object 404 Not Found – category does not exist or has no route yet
PUT	/api/categories/{categoryId}/route	Creates the route for a category, or replaces it if one exists (a category has at most one route, so PUT is idempotent).	Organiser	{ routeName, startLocation, finishLocation, distance_Km }	201 Created – route created 200 OK – existing route replaced 400 Bad Request – non-positive distance 403 Forbidden – not the owning organiser 404 Not Found – category does not exist
DELETE	/api/categories/{categoryId}/route	Removes the route from a category.	Organiser	None	204 No Content 403 Forbidden – not the owning organiser 404 Not Found – category has no route

6. Event Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments	Enters the logged-in Participant into an event by selecting a category. Records the link Participant → Category → Event (Enrolment.userId from session), sets paymentStatus = Pending and registeredAt = now; bibNumber is issued once paid.	Participant	{ categoryId }	201 Created – enrolment with enrolmentId, event, category, paymentStatus 400 Bad Request – categoryId missing 403 Forbidden – caller is an Organiser 404 Not Found – category does not exist 409 Conflict – already enrolled in this category, or event is not Open
GET	/api/enrolments/me	Lists all of the logged-in Participant's own enrolments across events, with event date, category, bib number, payment status and result (if any).	Participant	None	200 OK – array of enrolments 403 Forbidden – caller is an Organiser
GET	/api/enrolments/{id}	Returns one enrolment. Allowed for the Participant who owns it or the Organiser of the related event.	Any	None	200 OK – enrolment object 403 Forbidden – not the owner or event organiser 404 Not Found – enrolment does not exist
DELETE	/api/enrolments/{id}	Withdraws the Participant's own entry before race day. Refused once a	Participant	None	204 No Content 403 Forbidden – not the

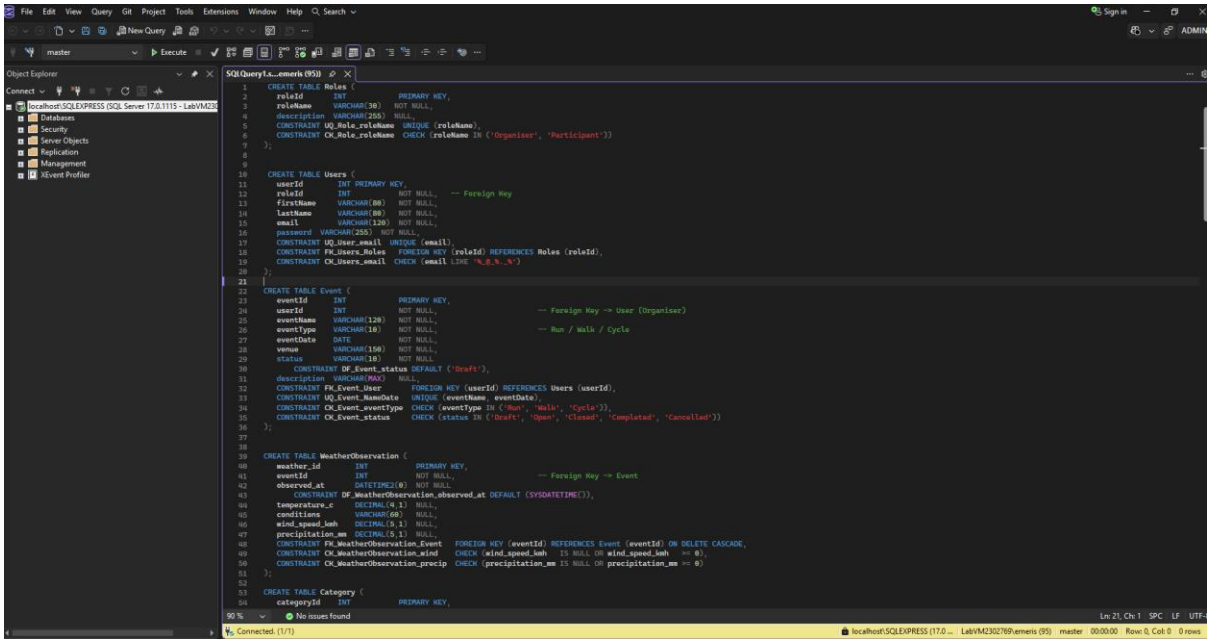
		result has been captured for it.			participant's own enrolment 404 Not Found – enrolment does not exist 409 Conflict – result already recorded
GET	/api/events/{eventId}/enrolments	Lets the Organiser view all enrolments for one of their events (participant name, category, bib number, payment status). Supports ?categoryId= and ?paymentStatus = filters.	Organiser	None	200 OK – array of enrolments 403 Forbidden – not the owning organiser 404 Not Found – event does not exist
PUT	/api/enrolments/{id}/payment	Updates the payment status of an enrolment (Pending → Paid / Refunded / Cancelled) and allocates the bibNumber when marked Paid.	Organiser	{ paymentStatus, bibNumber? }	200 OK – updated enrolment 400 Bad Request – invalid paymentStatus 403 Forbidden – not the owning organiser 404 Not Found – enrolment does not exist

7. Results

HTTP Method	Route	Description	Role required	Request body	Expected response
POST	/api/enrolments/{enrolmentId}/results	Captures a Participant's finish time and finishing positions after the event. Result.userId is set to the capturing Organiser from the session. resultStatus is Finished, DNF, DNS or DSQ; finishTime is required when Finished.	Organiser	{ finishTime, overallPosition, categoryPosition, resultStatus }	201 Created – result with resultId and recordedAt 400 Bad Request – invalid status/positions or missing finishTime 403 Forbidden – not the owning organiser

					404 Not Found – enrolment does not exist 409 Conflict – result already exists for this enrolment (use PUT)
PUT	/api/results/{id}	Corrects an existing result (e.g. adjusted time or position after a timing review).	Organiser	{ finishTime, overallPosition, categoryPosition, resultStatus }	200 OK – updated result 400 Bad Request – invalid field 403 Forbidden – not the owning organiser 404 Not Found – result does not exist
DELETE	/api/results/{id}	Removes a result captured in error.	Organiser	None	204 No Content 403 Forbidden – not the owning organiser 404 Not Found – result does not exist
GET	/api/results/me	Returns the logged-in Participant's own results across all events – their personal performance history (event, category, distance, finish time, positions).	Participant	None	200 OK – array of results ordered by eventDate 403 Forbidden – caller is an Organiser
GET	/api/results/{id}	Returns one result. Allowed for the Participant it belongs to or the Organiser of the event.	Any	None	200 OK – result object 403 Forbidden – not the owner or event organiser 404 Not Found – result does not exist
GET	/api/events/{eventId}/results	Public results list for a completed event, ordered by overallPosition; optional ?categoryId= filter for category standings.	None	None	200 OK – array of { participant, category, bibNumber, finishTime, overallPosition, categoryPosition, resultStatus } 404 Not Found – event does not exist

Section C



```
CREATE TABLE Roles (
    roleId      INT             PRIMARY KEY,
    roleName    VARCHAR(30)     NOT NULL,
    description VARCHAR(255)    NULL,
    CONSTRAINT UQ_Role_roleName  UNIQUE (roleName),
    CONSTRAINT CK_Role_roleName  CHECK (roleName IN ('Organiser', 'Participant'))
);

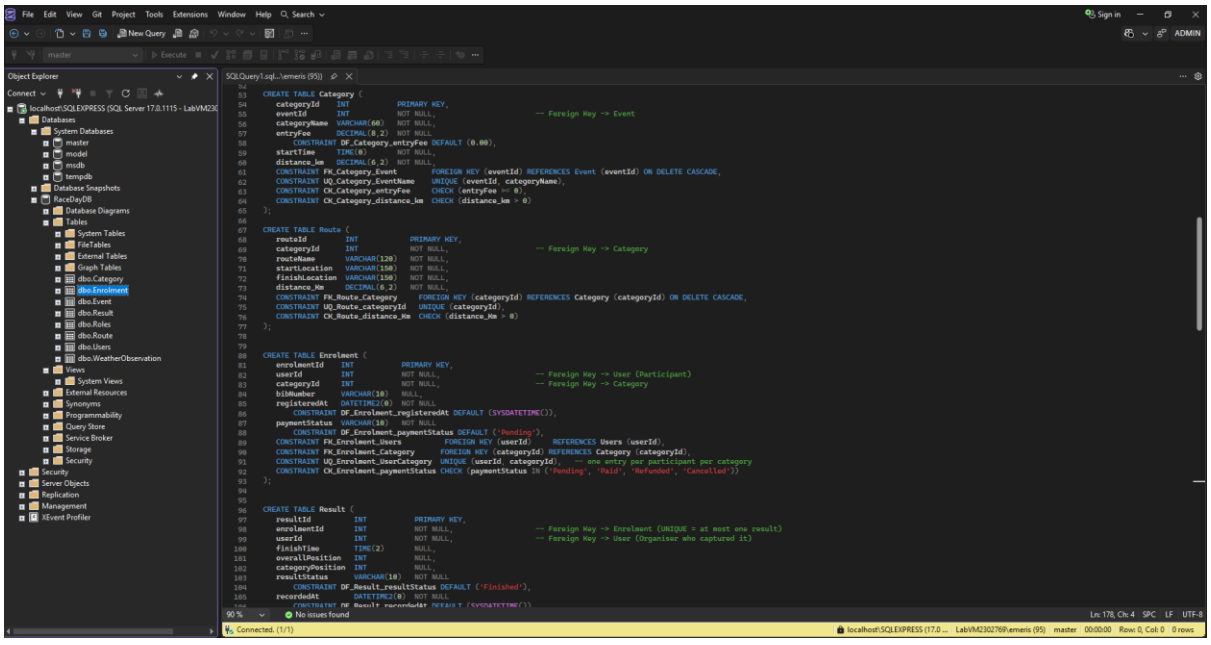

CREATE TABLE Users (
    userId      INT PRIMARY KEY,
    roleId      INT             NOT NULL,   -- Foreign Key
    firstName   VARCHAR(80)     NOT NULL,
    lastName    VARCHAR(80)     NOT NULL,
    email       VARCHAR(120)    NOT NULL,
    password    VARCHAR(255)    NOT NULL,
    CONSTRAINT UQ_User_email  UNIQUE (email),
    CONSTRAINT FK_Users_Roles   FOREIGN KEY (roleId) REFERENCES Roles (roleId),
    CONSTRAINT CK_Users_email   CHECK (email LIKE '%_@_%._%')
);

CREATE TABLE Event (
    eventId     INT                 PRIMARY KEY,
    userId      INT                 NOT NULL,                   -- Foreign Key -> User (Organiser)
    eventName   VARCHAR(120)        NOT NULL,
    eventType   VARCHAR(10)         NOT NULL,                   -- Run / Walk / Cycle
    eventDate   DATE                NOT NULL,
    venue       VARCHAR(150)        NOT NULL,
    status      VARCHAR(10)         NOT NULL
        CONSTRAINT DF_Event_status DEFAULT ('Draft'),
    description VARCHAR(MAX)        NULL,
    CONSTRAINT FK_Event_User        FOREIGN KEY (userId) REFERENCES Users (userId),
    CONSTRAINT UQ_Event_NameDate    UNIQUE (eventName, eventDate),
    CONSTRAINT CK_Event_eventType   CHECK (eventType IN ('Run', 'Walk', 'Cycle')),
    CONSTRAINT CK_Event_status      CHECK (status IN ('Draft', 'Open', 'Closed', 'Completed', 'Cancelled'))
);


CREATE TABLE WeatherObservation (
    weather_Id      INT             PRIMARY KEY,
    eventId         INT             NOT NULL,                   -- Foreign Key -> Event
    observed_at     DATETIME2(0)    NOT NULL
        CONSTRAINT DF_WeatherObservation_observed_at DEFAULT (SYSDATETIME()),
    temperature_c   DECIMAL(4,1)    NULL,
    conditions      VARCHAR(60)     NULL,
    wind_speed_kmh  DECIMAL(5,1)    NULL,
    precipitation_mm DECIMAL(5,1)   NULL,
    CONSTRAINT FK_WeatherObservation_Event   FOREIGN KEY (eventId) REFERENCES Event (eventId) ON DELETE CASCADE,
    CONSTRAINT CK_WeatherObservation_wind    CHECK (wind_speed_kmh   IS NULL OR wind_speed_kmh   >= 0),
    CONSTRAINT CK_WeatherObservation_precip  CHECK (precipitation_mm IS NULL OR precipitation_mm >= 0)
);

CREATE TABLE Category (
    categoryId  INT             PRIMARY KEY,
```

Figure 2: SQL code for creating the table (Microsoft, 2026)



```
CREATE TABLE Category (
    categoryId  INT             PRIMARY KEY,
    eventId     INT             NOT NULL,                   -- Foreign Key -> Event
    categoryName VARCHAR(60)    NOT NULL,
    entryFee    DECIMAL(8,2)    NOT NULL
        CONSTRAINT DF_Category_entryFee DEFAULT (0.00),
    startTime   TIME(0)         NOT NULL,
    distance_km DECIMAL(6,2)    NOT NULL,
    CONSTRAINT FK_Category_Event        FOREIGN KEY (eventId) REFERENCES Event (eventId) ON DELETE CASCADE,
    CONSTRAINT UQ_Category_EventName    UNIQUE (eventId, categoryName),
    CONSTRAINT CK_Category_entryFee     CHECK (entryFee >= 0),
    CONSTRAINT CK_Category_distance_km  CHECK (distance_km > 0)
);

CREATE TABLE Route (
    routeId     INT             PRIMARY KEY,
    categoryId  INT             NOT NULL,                   -- Foreign Key -> Category
    routeName   VARCHAR(120)    NOT NULL,
    startLocation  VARCHAR(150) NOT NULL,
    finishLocation VARCHAR(150) NOT NULL,
    distance_Km DECIMAL(6,2)    NOT NULL,
    CONSTRAINT FK_Route_Category    FOREIGN KEY (categoryId) REFERENCES Category (categoryId) ON DELETE CASCADE,
    CONSTRAINT UQ_Route_categoryId  UNIQUE (categoryId),
    CONSTRAINT CK_Route_distance_Km CHECK (distance_Km > 0)
);


CREATE TABLE Enrolment (
    enrolmentId INT         PRIMARY KEY,
    userId      INT         NOT NULL,                   -- Foreign Key -> User (Participant)
    categoryId  INT         NOT NULL,                   -- Foreign Key -> Category
    biNumber    VARCHAR(10) NULL,
    registeredAt DATETIME2(0) NOT NULL
        CONSTRAINT DF_Enrolment_registeredAt DEFAULT (SYSDATETIME()),
    paymentStatus VARCHAR(10) NOT NULL
        CONSTRAINT DF_Enrolment_paymentStatus DEFAULT ('Pending'),
    CONSTRAINT FK_Enrolment_Users       FOREIGN KEY (userId)    REFERENCES Users (userId),
    CONSTRAINT FK_Enrolment_Category    FOREIGN KEY (categoryId) REFERENCES Category (categoryId),
    CONSTRAINT UQ_Enrolment_UserCategory  UNIQUE (userId, categoryId),  -- one entry per participant per category
    CONSTRAINT CK_Enrolment_paymentStatus CHECK (paymentStatus IN ('Pending', 'Paid', 'Refunded', 'Cancelled'))
);


CREATE TABLE Result (
    resultId        INT         PRIMARY KEY,
    enrolmentId     INT         NOT NULL,                   -- Foreign Key -> Enrolment (UNIQUE = at most one result)
    userId          INT         NOT NULL,                   -- Foreign Key -> User (Organiser who captured it)
    finishTime      TIME(2)     NULL,
    overallPosition INT         NULL,
    categoryPosition INT        NULL,
    resultStatus    VARCHAR(10) NOT NULL
        CONSTRAINT DF_Result_resultStatus DEFAULT ('Finished'),
    recordedAt      DATETIME2(0) NOT NULL
```

Figure 3: SQL code for creating the table (Microsoft, 2026)

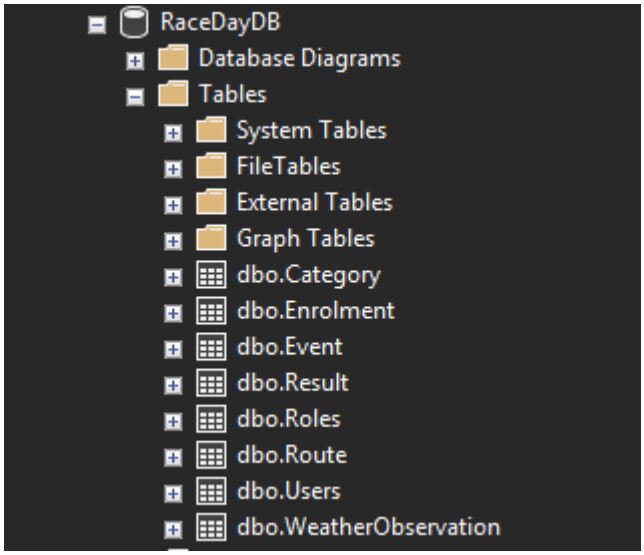


Figure 4: Tables under the RaceDayDB (Microsoft, 2026)

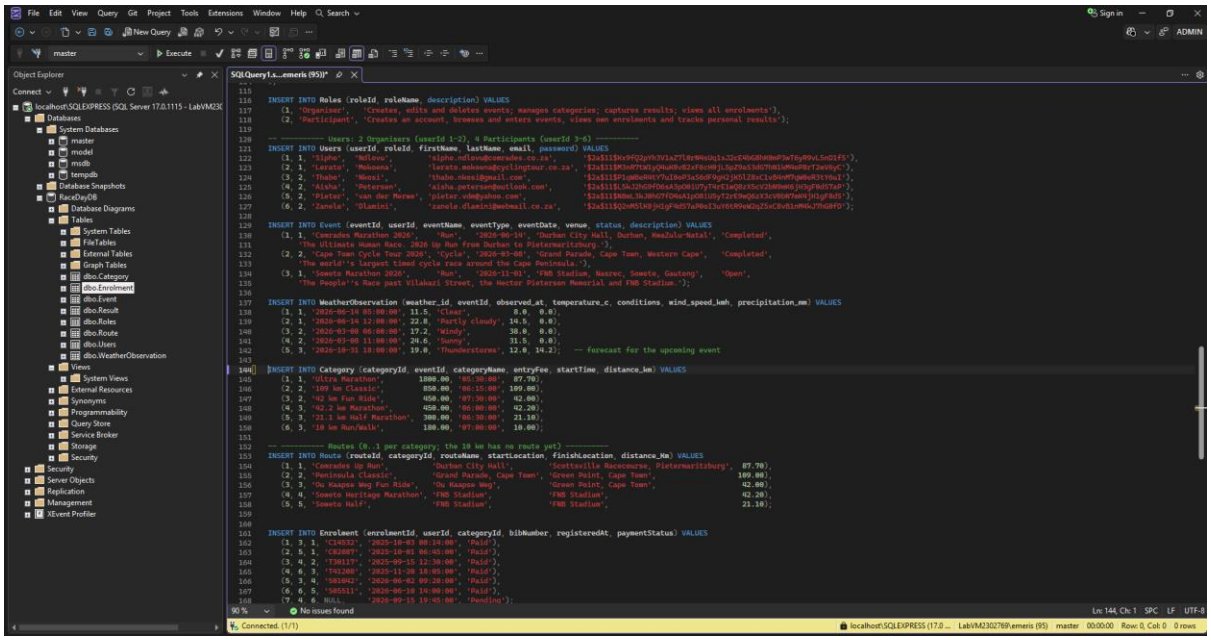


Figure 5: SQL code for inserting data in the tables (Microsoft, 2026)

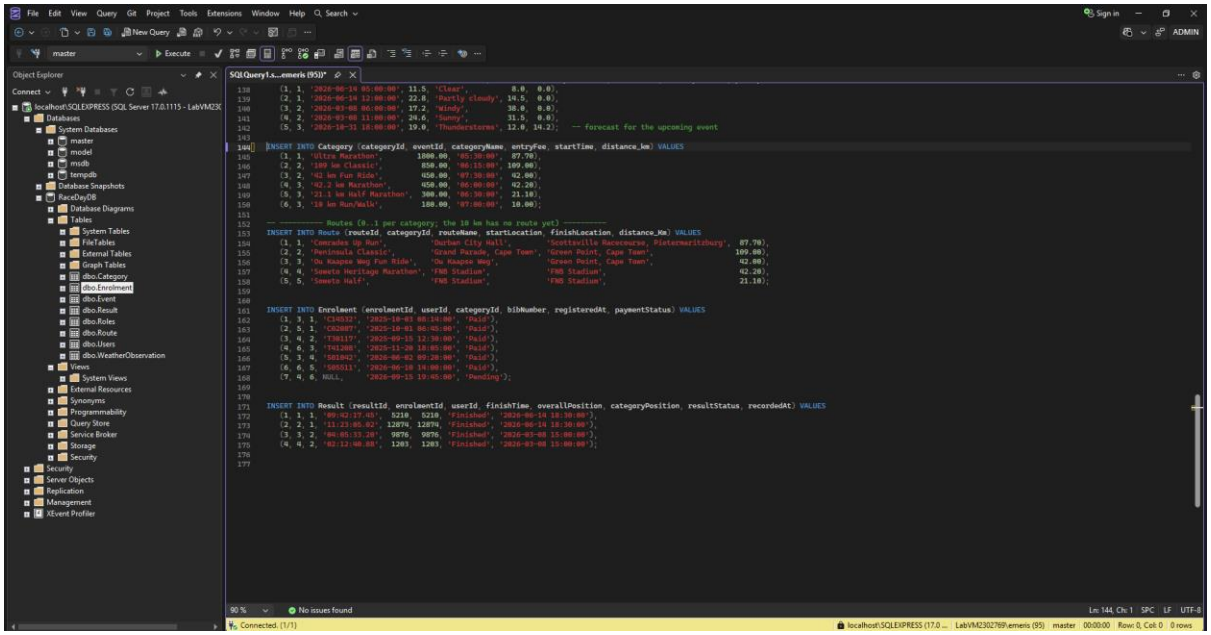


Figure 6: SQL code for inserting data in the tables (Microsoft, 2026)

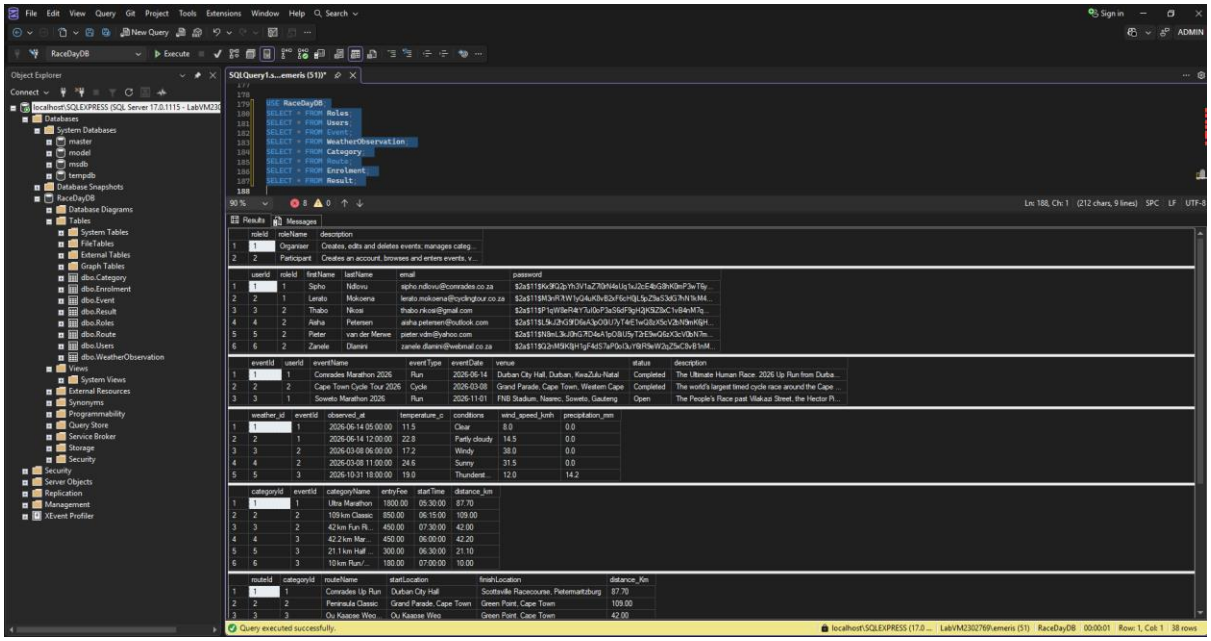


Figure 7: The inserted data (Microsoft, 2026)

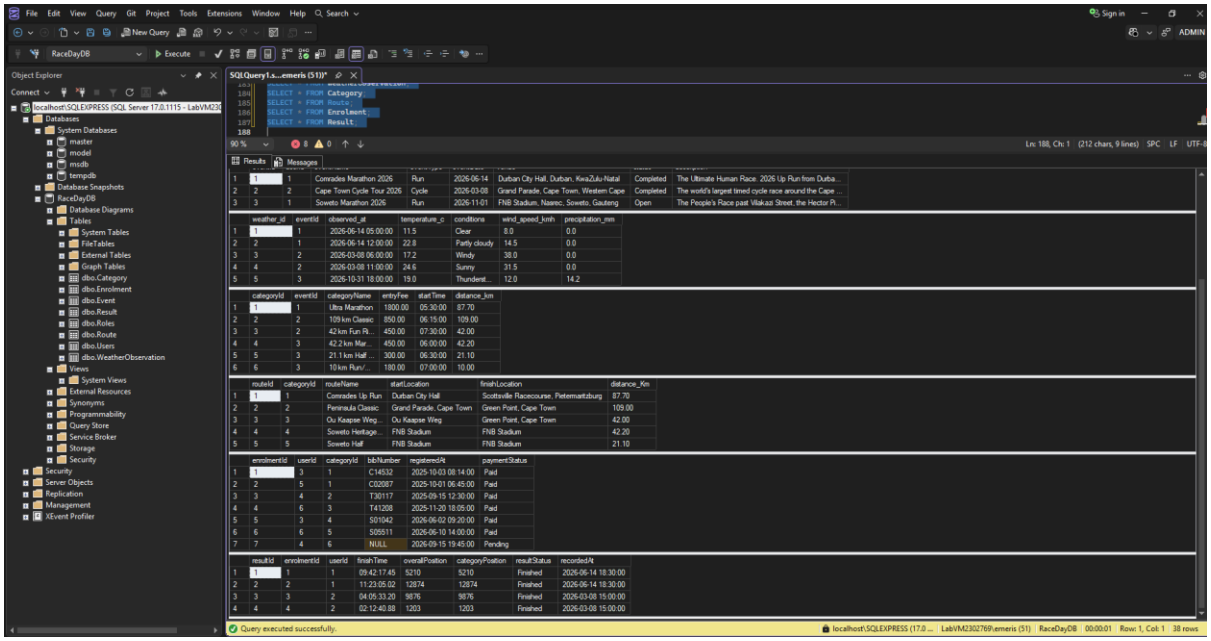


Figure 8: The inserted data (Microsoft, 2026)

GitHub link

[EMGPSD/emgpsd-prog6212-g3-2026-poe-part1-st10441596]

Referencing

Lucid Software Inc. 2026. Lucidchart. [Online]. Available at: <https://www.lucidchart.com >[Accessed 20 September 2026].

Microsoft, 2025. *CREATE TABLE (Transact-SQL)*. [Online]. Available at: <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-table-transact-sql >[Accessed 20 September 2026].

Microsoft, 2025. *Data types (Transact-SQL)*. [Online]. Available at: <https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql >[Accessed 20 September 2026].

Microsoft, 2025. *Primary and foreign key constraints*. [Online]. Available at: <https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints >[Accessed 20 September 2026].

Microsoft, 2026. *SQL Server Management Studio (SSMS)*. [Online]. Available at:<https://learn.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms >[Accessed 20 September 2026].

Mozilla Developer Network. 2026. *HTTP response status codes*. [Online]. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status >[Accessed 22 September 2026].